

H1 freight optimisation: what happened at each step

This is a step-by-step account of one run, written from its trace (179 lines), its step outputs and its workspace files. It covers the first of the three hard probe tasks; the summary for all three is in [REPORT.md](#).

- **Run:** 20a96784-9439-4ae5-8909-20e7cb80fa3d , 4 October 2026, 17:26–17:57 UTC.
- **Evidence:** eval/probe_hard/runs/20a96784... on the evidence branch (commit 36dedf2).
- **Code:** PR #34 head 6423a95 . Gemma 4 31B was the only model.
- **Flags:** --topology plan --routing routed --niche general --local-tools on --local-tools-mode sandbox --web-tools --replan on --draft-prompts d24 .

The task (verbatim). A carrier must move 10 loads today using up to 4 trucks. Each load goes on exactly one truck. A truck that carries any load costs its fixed daily cost plus its per-mile rate times the total miles of its loads. Constraints per truck: total weight $\leq$ weight capacity, total miles $\leq$ mileage limit.

- Loads (weight lb, miles): L1 18000, 220; L2 12500, 340; L3 22000, 150; L4 9000, 410; L5 15500, 95; L6 7000, 280; L7 20500, 190; L8 11000, 365; L9 14000, 120; L10 8500, 255.
- Trucks (capacity lb, mileage limit, fixed \$/day, \$/mile): A 45000, 1100, 350, 2.10; B 45000, 1100, 350, 2.10; C 30000, 1100, 500, 1.85; D 26000, 900, 275, 2.45.

Find the minimum-cost assignment. Give the total cost, each truck's loads, weight and miles, and show how you know it is optimal. Deliver an .xlsx with the plan and the cost calculation as formulas.

Timeline

Minutes	Box	What happened	Model calls
0.0-0.3	1 · Task	Family chosen by rule; one interpretation call	1
0.3-8.9	2 · Plan a team	Two draft rounds, each critiqued by two reviewers; plain-code quality gate between them	6
8.9-9.5	Toolbox	Sandbox created; tools matched to the three capability requests	2
9.5-13.4	Step 1 · Solve	Optimization Engineer, 4 turns	4
13.4-13.8	Re-plan check	Action Observer, 1 call: CONTINUE	1
13.8-22.9	Step 2 · Verify	QA Engineer, 3 blind turns + 4 checking turns	7
22.9-27.0	Step 3 · Spreadsheet	Document Specialist, 3 turns	3
27.0-30.8	Step 4 · Summary	Summariser, 2 turns	2
30.8-30.9	Wrap-up	Requirement status, figure ledger, sandbox deleted	0

Total: 30.9 minutes and 27 model calls. Tokens: 92,781 in, 26,923 out, plus 36,542 hidden reasoning tokens (156,246 billed). Every call went to `gemma-4-31b`. There were 4 rate-limit waits (2 s, 4 s, 8 s, 16 s), all in step 1, because M-P2 was sharing the API key.

Box 1: understanding the task

1. **Family.** Plain code gave the task the family `calc`, because the prompt contains the keywords "total" and "cost". No AI was involved in this choice.
2. **Interpretation.** One call (651 tokens in, 89 out) read the task for ambiguous names, acronyms or terms with two readings. It found none. "lb", "\$/mile" and ".xlsx" were judged standard. So the run had no entities to settle and no clarification question.

Box 2: planning the team

Requirements. Both reviewers split the task into the same seven requirements:

Id	Requirement
R1	the minimum-cost assignment
R2	the total cost
R3	each truck's loads
R4	each truck's weight and miles
R5	proof of optimality
R6	an .xlsx file
R7	formulas in the .xlsx

Round 1 (planner call, then the staffing reviewer and the plan reviewer).

- **Team proposed:** Optimization Engineer, QA Engineer and Delivery Lead.
- **Both reviewers** found every requirement covered.
- **Plan reviewer's own check:** it re-added the task's figures (total weight 138,000 lb, total miles 2,425, total capacity 146,000 lb), and all three were right.
- **Rejected by plain code:** the quality gate failed the draft on the **summariser** rule. The Delivery Lead was both the summariser and the person building the spreadsheet with the `xlsx` skill. The summariser must be a separate role with no tools, so the draft went back to the planner.

Round 2. The planner split the Delivery Lead into a **Document Specialist** (builds the `.xlsx`) and a **Summariser** (no tools). Both reviewers agreed, so the draft reached consensus after 2 rounds.

The final team:

Role	Job	Tools asked for	Covers
Optimization Engineer (senior)	Find the proven minimum with a script	<code>python_interpreter</code>	R1-R5
QA Engineer	Check the assignment and the arithmetic independently	<code>calc</code>	R1-R4
Document Specialist	Build <code>assignment_plan.xlsx</code> with formulas	<code>python_interpreter</code> , <code>xlsx</code> skill	R6-R7
Summariser	Assemble the final answer	none	R1-R7

The plan, in four waves that each depend on the ones before:

1. **Solve** (Optimization Engineer). Write and run a Python script that searches every valid load-to-truck assignment and keeps the cheapest. Done when the script finds the global minimum.
2. **Verify** (QA Engineer). Re-add each truck's weight and miles, check them against the limits, re-compute the cost, and confirm all 10 loads are present.
3. **Build the spreadsheet** (Document Specialist). Use Python to write `assignment_plan.xlsx`, with each truck's cost ($\text{fixed} + \text{rate} \times \text{miles}$) as a formula.
4. **Summary** (Summariser). Total cost, assignment, truck figures, proof, and the file.

Draft quality (plain code). The draft passed 7 of 8 checks:

- **Passed:** every requirement covered by a role and a step; valid dependencies; every role fully defined; exactly one summariser, with no tools; a verification step; and that verifier independent of the producer.
- **Failed:** `tools_accounted`. The roles needed `python_interpreter`, but the planner had not listed it in its own tool requests. This is logged, not blocking.

Toolbox: getting the tools

- **Sandbox.** A fresh OpenShell sandbox (`am-60923a6724bbfb`) was created with no network, 1 CPU and 1 GiB.
- Inside it, `claude mcp serve` offered the local tools.
- 17 Claude Code tools outside the allowlist (Agent, WebFetch, Skill and others) were turned off, as designed.

- `Glob` and `Grep` were not offered by that version.
- **Tool matching.** The pool (35,703 entries) matched all three requests.
- `xlsx` → the local `anthropics_skills/xlsx` skill. Attaching it was then **refused**: "not in the sandbox image". This was problem 1 in the report, fixed by D103.
- `python_interpreter` → `local:Bash`, for the Optimization Engineer and the Document Specialist. One picker call per role vetted it.
- **Verifier tools.** The QA Engineer, as verifier, was later also given `web_search`, `fetch_url`, `local:Bash` and `local:Read`, so it could re-run code (D65).

Step 1: Solve (Optimization Engineer, 4 turns, 3.9 minutes)

- **Turn 1.** The helper wrote a Python exhaustive search and sent it to `local:Bash` as raw Python. Bash cannot run Python directly, so the call failed: `import: command not found`, exit code 2. This was problem 7 in the report, fixed by D112, which runs such input with `python3`.
- **Turn 2.** It noticed the error and wrapped the same script in a here-doc (`cat << 'EOF' > solution.py`), then ran it with `python3`.
- **The script:** `itertools.product(range(4), repeat=10)` walks through all $4^{10} = 1,048,576$ ways to put 10 loads on 4 trucks. For each, it adds up every truck's weight and miles, throws out any assignment that breaks a limit, prices the trucks that are used, and keeps the cheapest.
- **Run time:** 8.0 s in the sandbox.
- **Output** (recorded as source [S1]): Minimum Cost: 6356.25 Truck A: Loads=['L1', 'L2', 'L9'], Weight=44500, Miles=680 Truck B: Loads=['L5', 'L7', 'L10'], Weight=44500, Miles=540 Truck C: Loads=['L4', 'L6', 'L8'], Weight=27000, Miles=1055 Truck D: Loads=['L3'], Weight=22000, Miles=150
- **Turn 3** (after four rate-limit waits). It re-did each truck's cost by hand:
 - $A \ 350 + 2.10 \times 680 = 1,778$
 - $B \ 350 + 2.10 \times 540 = 1,484$
 - $C \ 500 + 1.85 \times 1,055 = 2,451.75$
 - $D \ 275 + 2.45 \times 150 = 642.50$

The total, 6,356.25, matched. It then gave its answer. - **Format check.** Plain code rejected that answer: the step promises a memo, and a memo needs at least two headings. The step got a retry. - **Turn 4.** The same content came back with headings: Summary, Detailed Truck Assignments, Proof of Optimality. It also included the line "NOT NEEDED: local:Bash — the script has already been run". That passed. - **Checks after the step:** - Contract: nothing missing; 2 tool calls, 1 of them successful. - Provenance: 14 figures, all cited, none invented. - **Citation check:** it flagged "1,048,576 [S1]" as a mislabelled citation, because that number is not in the script's output; the helper calculated it as 4^{10} . This is the only reason step 1 was marked **partial**. It is a false alarm, fixed by D104, which lets the check accept in-line arithmetic.

Re-plan check (Action Observer, 1 call)

- **Why it ran.** Plain code paused after wave 1 because something had changed: "Document Specialist now holds local:Bash, which the plan never named". The planner had asked for `python_interpreter`, and the pool met that request with `local:Bash`.
- **Decision.** The observer judged that step 1 had produced everything the later steps need. It answered **CONTINUE**, with no new steps, roles or capability requests.
- **Validation.** Plain code accepted the decision.

Step 2: Verify (QA Engineer, 7 turns, 9.1 minutes)

Part 1: its own answer first, without seeing step 1 (verify-first, D90). Step 1's output was hidden from the helper.

- **Turn 1.** Same mistake as step 1: it sent raw Python to Bash, and the call failed.
- **Turn 2.** It wrote `solution.py` with a here-doc and ran it.
- **The script:** a different method from step 1. It is a depth-first search that places one load at a time and skips any truck that would go over its weight or miles. So it prunes, instead of listing every combination.
- It **overwrote** step 1's `solution.py` (same name, same workspace).
- **Output [S2]:** minimum 6,356.25, assignment A A D C B C B C A B (one letter per load, L1 to L10), with the same per-truck weights, miles and costs as step 1.
- **Turn 3.** It re-added all four trucks by hand and confirmed the limits and the 10 loads. Its own answer: 6,356.25, with the same assignment.

Part 2: checking step 1 (step 1's output now shown).

- **Turn 1.** It sent a multi-line Python program to the `calc` tool. `calc` only takes one expression, so it answered `invalid syntax`.
- **Turn 2.** It wrote `verify.py`, which checks the claimed assignment against the raw task data, and ran it with `python3`.
- **Output [S3]:** each truck within its limits, costs 1,778.00 / 1,484.00 / 2,451.75 / 642.50, total 6,356.25, "All loads assigned: True", "Load count: 10".
- **Turn 3.** It drafted the verification report.
- **Turn 4.** Plain code found one figure without a source tag. The helper retagged it, and the report passed.

Verdict. PASS, issues: none. Its own blind answer and step 1's answer were identical, so there was no disagreement to settle.

Checks after the step:

- Provenance: 85 numbers (65 cited, 18 given in the task, 2 derived).
- Contract: 4 tool calls, 2 successful.
- **Citation check:** it flagged 4 mislabelled citations. These are lines such as " $350 + (2.10 \times 680) = 350 + 1,428.00 = 1,778.00$ [S3]", where the intermediate 1,428.00 is not in the script output. So the step was marked **partial**, although its verdict was PASS. Same false alarm as step 1, fixed by D104.

Step 3: Build the spreadsheet (Document Specialist, 3 turns, 4.1 minutes)

- **Skill.** The `xlsx` skill was not available, because the sandbox refused it. The helper wrote the file with `openpyxl` instead.
- **Turn 1.** Raw Python sent to Bash again, and the call failed (the third helper to do this).
- **Turn 2.** It wrote `create_excel.py` with a here-doc and ran it (5.0 s, exit code 0, no output). That produced `assignment_plan.xlsx` (5,657 bytes) [S4]. The workbook has one sheet, "Assignment Plan", with four tables: 1. Loads (A1:C11): the 10 loads with their weight and miles, typed in. 2. Trucks (A13:E17): capacity, mileage limit, fixed cost and rate, typed in. 3.

Assignment (A19:D23): each truck's loads, with its total weight and miles **typed in as numbers** (44500 / 680 ...), not summed from the loads table. 4. Cost (A25:E30): fixed cost =D14 , rate =E14 , miles =D20 , truck cost =B26+(C26*D26) for each truck, and the total =SUM(E26:E29) , which works out to 6,356.25.

- **Turn 3.** It described the file and gave its answer.
- **Checks after the step:** contract met, with "Print" declared not needed. Provenance: 10 numbers, all cited. Status **done**.

Verdict on the spreadsheet: partly right. The costs and the total are real formulas. But the per-truck weights and miles are typed in, so changing a load in the loads table would not update them. D113 now checks for exactly this.

Step 4: Summary (Summariser, 2 turns, 3.8 minutes)

- **Turn 1.** It drafted the final report.
- **Contract check.** Plain code found two gaps in the draft:
 - it did not name the files the team had made;
 - it left out figures from [S3] that the task needs: the truck limits, the rates and each truck's cost.
- **Turn 2.** The rewrite added a table with each truck's limits and cost, and a Deliverables list. One finding was left after the rewrite.
- **Code checks on the summary:**
 - no new number appears in it that was not in a step output;
 - it has a Limitations section;
 - its 24 figures are all cited.
- **Status. Partial**, for the same "1,048,576 [S1]" citation flag as step 1.
- **Requirements.** Plain code then recorded the final requirement status: R6 and R7 met, R1-R5 "partly". R1-R5 are "partly" only because steps 1 and 2 were marked partial for the citation flags.

The final answer (as delivered)

Total Minimum Cost: 6,356.25

Truck	Loads	Total Weight (Limit)	Total Miles (Limit)	Cost
A	L1, L2, L9	44,500 lb (45,000)	680 (1,100)	1,778.00
B	L5, L7, L10	44,500 lb (45,000)	540 (1,100)	1,484.00
C	L4, L6, L8	27,000 lb (30,000)	1,055 (1,100)	2,451.75
D	L3	22,000 lb (26,000)	150 (900)	642.50

Proof of optimality: a Python script evaluated all 4^{10} (1,048,576) possible assignments, checked each truck's weight and miles against its limits, priced every valid one, and kept the cheapest.

Deliverables: assignment_plan.xlsx (plan and cost formulas), solution.py, verify.py, create_excel.py.

Limitations: steps 1 and 2 marked partial for mislabelled citations.

(Source tags removed here for reading.)

Against the checkpoints

Checkpoint	Result
Optimum \$6,356.25	Right
Assignment A: L1,L2,L9 / B: L5,L7,L10 / C: L4,L6,L8 / D: L3	Right , with the correct weight, miles and cost for each truck
All four trucks used	Right . The answer does not give the short argument (138,000 lb of loads > 120,000 lb in any three trucks); the full search covers it
Solver written and run in the sandbox	Right : two independent solvers, in steps 1 and 2
xlsx computes the total	Partly : costs and total are formulas; truck weights and miles are typed in
Verifier recomputed it blind	Right

Small things the run got wrong

1. **Raw Python sent to Bash** by all three helpers that used it (3 wasted turns). Fixed by D112.
2. **False citation flags** on numbers the helpers calculated in-line (1,048,576 and four intermediates). These made steps 1, 2 and 4 "partial" and R1-R5 "partly", although every figure was right. Fixed by D104.
3. **xlsx skill refused** by the sandbox. Fixed by D103.
4. **Typed-in weights and miles** in the spreadsheet. Now checked by D113.
5. **File overwritten**. The verifier reused the name `solution.py`, so step 1's own script is no longer in the workspace. Only its output (in the trace) remains.
6. **Wrong source tags in the Deliverables list**. It tags `verify.py` [S2] (it is [S3]) and `create_excel.py` [S3] (it is [S4]). No check looks at tags on file names.
7. **Wrong tool for the job**. The verifier sent a multi-line program to `calc`, which takes a single expression.
8. **No currency sign**. The total is written without "\$".

None of these changed the answer.